Module 3 : Structures de données

Situations

Caractères & Codes

On donne le programme suivant :

```python
car = "E"
num = ord(car)
print("Caractère :", chr(num))
print("Code :", ord(car))
```

1. Copier/coller, puis tester le programme. Qu'est ce qu'il affiche ?
2. En déduire le rôle des fonctions `chr(num)` et `ord(car)`.
3. En utilisant ce programme, déterminer les codes ASCII des caractères suivants : "e", "4"
4. En utilisant ce programme, déterminer les caractères correspondants aux code ASCII suivants : 90, 33
5. En se référant à la table ASCII suivante, trouver les codes ASCII :
   - des chiffres : 0, 1, ..., 9
   - des lettres alphabétiques majuscules : A, B, ..., Z
   - des lettres alphabétiques minuscules : a, b, ..., z

Exercice

Ecrire l'algorithme d'un programme qui convertit une lettre majuscule en minuscule en utilisant les fonctions `chr(code)` et `ord(car)`.
Traduire l'algorithme en Python.
Quelle est la modification à apporter à cet algorithme pour convertir une lettre minuscule en majuscule ?

Nombre automorphe

Un **nombre automorphe** est un entier naturel dont la suite des chiffres du carré se termine par celle du nombre lui-même. Par exemple,

$$5^2 = 25,\ 6^2 = 36,\ 25^2 = 625,$$

$$76^2 = 5776,\ 890625^2 = 793212890625$$

On veut écrire l'algorithme d'un programme qui teste si un nombre est automorphe.
On demande de :

1. Déterminer, d'après **la définition d'un nombre automorphe**, quels sont les nombres automorphes dans la liste suivante : 376, 9376, 17, 125.
2. Afin d'écrire l'algorithme du programme, nous devons répondre aux questions suivantes :
   - Comment calculer le carré d'un nombre ?
   - Comment tester si le nombre est automorphe ?
   - Comment convertir un nombre en une chaîne ?
   - Comment trouver la longueur du nombre contenu dans une chaîne ?
   - Comment extraire les deniers chiffres d'une chaîne ?
3. Ecrire l'algorithme, puis le traduire en Python.

Le plus âgé

On veut faire un programme qui détermine si une personne est plus âgée qu'une autre. Pour cela on donne le programme ci-dessous :

Python

```python
nom1 = input("Nom de la 1ère personne ? ")
age1 = input("Âge de " + nom1 + " ? ")

nom2 = input("Nom de la 2ème personne ? ")
age2 = input("Âge de " + nom2 + " ? ")

c1 = age1 > age2

print(nom1, "est plus âgé que", nom2, "?", c1)
```

On demande de :

1. Retrouver l'algorithme du programme, et dresser son **tableau de déclaration des objets (TDO)**.
2. Tester le programme à l'aide des valeurs suivantes. Est-il correct ? Quel est le problème ? Comment le corriger ?

nom1	age1	nom2	age2
Abderrazek	9	Hala	3
Emna	73	Abderrazek	9

Pomme ou pomme de terre ?

Pour son cours de français, Ahmed a rédigé le paragraphe suivant, un peu distrait par son petit frère il a commis deux types d'erreurs :

- Il a ajouté des mots qui devront être ~~supprimés~~ pour rectifier son texte.
- Il a oublié des mots qui devront être ajoutés pour enrichir son texte.

Les bienfaits de la pomme ~~de terre~~

La pomme ~~de terre~~ est un fruit succulent qui apporte une grande satiété, se transporte facilement et peut se consommer partout.
La pomme ~~de terre~~ est particulièrement intéressante pour les sportifs : avant l'effort elle apporte de l'énergie, pendant l'effort, elle apporte ~~des minéraux~~ des minéraux et des vitamines qui rechargent l'organisme et après l'effort, elle réhydrate !

Voulant montrer ses nouveaux acquis en programmation Python, Ahmed a essayé d'écrire l'algorithme suivant :

Algorithme

```
Algorithme Pomme_de_terre
Début
  titre ← "Les bienfaits de la pomme de terre"
  ph1 ← "La pomme de terre est un fruit qui apporte une grande satiété," +
        " se transporte facilement et peut se consommer partout."
  ph2 ← "La pomme de terre est particulièrement intéressante :" +
        " avant l'effort elle apporte de l'énergie," +
        " pendant l'effort, elle apporte des minéraux des minéraux" +
        " et des vitamines qui rechargent l'organisme" +
        " et après l'effort, elle réhydrate !"

  // Supprimer les mots " de terre"
  // du titre, de ph1 et de ph2
```

```
  // Supprimer les mots " des minéraux" de ph2

  // Insérer
  // le mot "succulent" après "est un fruit" dans ph1
  // les mots "pour les sportifs" après "intéressante" dans ph2

  Ecrire(titre)
  Ecrire(ph1)
  Ecrire(ph2)
Fin
```

Afin de compléter son algorithme, Ahmed se demande :

1. Comment retrouver la 1e position d'un mot dans une chaîne ?
2. Comment effacer un mot d'une chaîne ?
3. Comment insérer un mot dans une chaîne ?

On demande de compléter l'algorithme précédent. Puis, de le traduire en Python.

Le Halloween & les Bonbons

A l'occasion du Halloween, Samer et ses deux soeurs aînées Sarra at Samar se déguisent pour le Halloween. Le soir, ils visitent les maisons du quartier tout en chantant. Les voisins, très reconnaissants pour le geste, remplissent leur citrouille de bonbons et de friandises.
À la fin du soirée, les trois enfants à la maison, partagent leur butin. Chaque enfant doit avoir le même nombre de bonbons, le reste est partagé entre la mère et le père.
Pour une distrubtion équitable des friandises leur père a écrit l'algorithme' suivant :

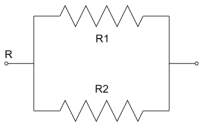
Algorithme

```
Algorithme halloween
Début
  nb ← 16 // Nombre total de bonbons
  nbe ← 3 // Nombre d'enfants

  nbpe ← 5 // Nombre de bonbons par enfant
  nbr ← 1  // Nombre de bonbons restants

  Ecrire("Nbre de bonbons par enfant :", nbpe)
  Ecrire("Nbre de bonbons restants :", nbr)
Fin
```

On demande de :

1. Dresser le TDQ de cet algorithme.
2. Modifier l'algorithme pour qu'il :
   - accepte le nombre total de bonbons, ainsi que le nombre d'enfants.
   - calcule le nombre de bonbons par enfant, ainsi que le nombre de bonbons restants.
3. Traduire l'algorithme en Python.

Résistance équivalente



$$R_{eq} = \frac{R_1 R_2}{R_1 + R_2}$$

Figure 1, Résistance équivalente de deux résistances montés en parallèle

On demande de :

1. Calculer la résistance équivalente de deux résistances dans les cas suivants :

r1 (Ω)	r2 (Ω)
0.1	0.2
470	220

2. D'écrire l'algorithme de ce programme. Puis, de dresser son TDO.
3. Traduire l'algorithme en Python, puis tester le programme avec les valeurs du tableau précédent.
4. Améliorer le programme pour qu'il affiche des résultats à 2 déciamles : `Req = 66.67 Ω` au lieu de `Req = 66.66666666666667 Ω`.

Logic gates

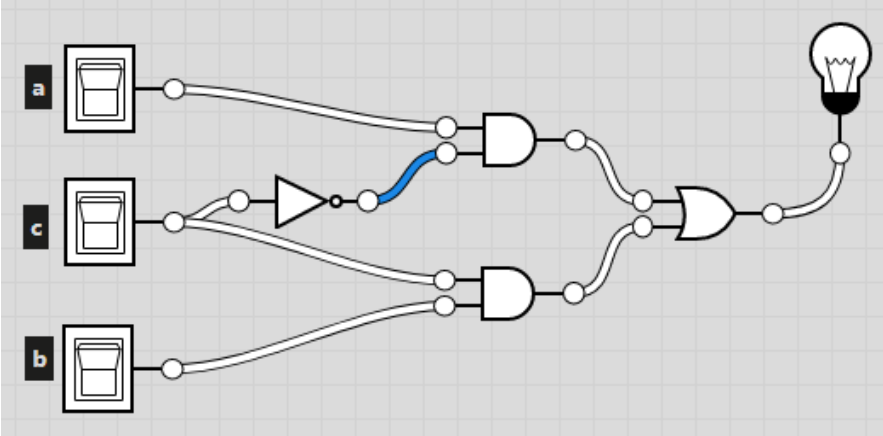


Figure 2, Portes logiques

1. Soit le circuit logique ci-dessus, on demande d'écrire l'équation logique de la lampe `L` en fonction des entrées logiques `a`, `b` et `c`.
2. Dresser le tableau de vérité de ce circuit. Puis, remplir la colonne L.

a	b	c	L
Faux	Faux	Faux	
Faux	Faux	Vrai	
Faux	Vrai	Faux	
Faux	Vrai	Vrai	
Vrai	Faux	Faux	
Vrai	Faux	Vrai	
Vrai	Vrai	Faux	
Vrai	Vrai	Vrai	

3. On donne l'algorithme incomplet suivant qui permet de simuler ce circuit :

Python

```
Algorithme Logic_gates
Début
  // etat est une chaîne contenant trois lettres
  // "V" bouton fermé / "F" bouton ouvert
  Ecrire("Etat des boutons") ; Lire(etat)

  a ← Faux // Vrai si le 1e bouton est fermé
  b ← Faux // Vrai si le 2e bouton est fermé
  c ← Faux // Vrai si le 3e bouton est fermé

  L ← Faux // Equation logique de L

  // Résultat
  Ecrire(a, ",", b, ",", c, "=>", L)
Fin
```

4. Dresser le TDO du programme.
5. Compléter l'algorithme, le traduire en Python, puis le tester.
6. Améliorer l'algorithme et le programme pour qu'ils acceptent des lettres majuscules ou minuscules.

Résumé

Type Caractère

Un caractère peut-être :

- Une lettre alphabétique majuscule : "A", "B", ..., "Z"
- Une lettre alphabétique minuscule : "a", "b", ..., "z"
- Un chiffre : "0", "1", ..., "9"
- Un symbole : " ", "!", "\"", "#", etc.

Chaque caractère possède un code normalisé appelé :

- **Code ASCII** pour les 256 premiers caractères (voir table ASCII)
- **Code Unicode** au dela des 256 premiers caractères.

ASCII control characters		
00	NULL	(Null character)
01	SOH	(Start of Header)
02	STX	(Start of Text)
03	ETX	(End of Text)
04	EOT	(End of Trans.)
05	ENQ	(Enquiry)
06	ACK	(Acknowledgement)
07	BEL	(Bell)
08	BS	(Backspace)
09	HT	(Horizontal Tab)
10	LF	(Line feed)
11	VT	(Vertical Tab)
12	FF	(Form feed)
13	CR	(Carriage return)
14	SO	(Shift Out)
15	SI	(Shift In)
16	DLE	(Data link escape)
17	DC1	(Device control 1)
18	DC2	(Device control 2)
19	DC3	(Device control 3)
20	DC4	(Device control 4)
21	NAK	(Negative acknowl.)
22	SYN	(Synchronous idle)
23	ETB	(End of trans. block)
24	CAN	(Cancel)
25	EM	(End of medium)
26	SUB	(Substitute)
27	ESC	(Escape)
28	FS	(File separator)
29	GS	(Group separator)
30	RS	(Record separator)
31	US	(Unit separator)
127	DEL	(Delete)

ASCII printable characters					
32	space	64	@	96	`
33	!	65	A	97	a
34	"	66	B	98	b
35	#	67	C	99	c
36	$	68	D	100	d
37	%	69	E	101	e
38	&	70	F	102	f
39	'	71	G	103	g
40	(	72	H	104	h
41	)	73	I	105	i
42	*	74	J	106	j
43	+	75	K	107	k
44	,	76	L	108	l
45	-	77	M	109	m
46	.	78	N	110	n
47	/	79	O	111	o
48	0	80	P	112	p
49	1	81	Q	113	q
50	2	82	R	114	r
51	3	83	S	115	s
52	4	84	T	116	t
53	5	85	U	117	u
54	6	86	V	118	v
55	7	87	W	119	w
56	8	88	X	120	x
57	9	89	Y	121	y
58	:	90	Z	122	z
59	;	91	[	123	{
60	<	92	\	124	|
61	=	93	]	125	}
62	>	94	^	126	~
63	?	95	_		

Extended ASCII characters							
128	Ç	160	á	192	└	224	Ó
129	ü	161	í	193	┴	225	ß
130	é	162	ó	194	┬	226	Ô
131	â	163	ú	195	├	227	Ò
132	ä	164	ñ	196	─	228	õ
133	à	165	Ñ	197	┼	229	Õ
134	å	166	ª	198	ã	230	µ
135	ç	167	º	199	Ã	231	þ
136	ê	168	¿	200	╚	232	Þ
137	ë	169	®	201	╔	233	Ú
138	è	170	¬	202	╩	234	Û
139	ï	171	½	203	╦	235	Ù
140	î	172	¼	204	╠	236	ý
141	ì	173	¡	205	═	237	Ý
142	Ä	174	«	206	╬	238	¯
143	Å	175	»	207	¤	239	´
144	É	176	░	208	ð	240	≡
145	æ	177	▒	209	Ð	241	±
146	Æ	178	▓	210	Ê	242	‗
147	ô	179	│	211	Ë	243	¾
148	ö	180	┤	212	È	244	¶
149	ò	181	Á	213	ı	245	§
150	û	182	Â	214	Í	246	÷
151	ù	183	À	215	Î	247	¸
152	ÿ	184	©	216	Ï	248	°
153	Ö	185	╣	217	┘	249	¨
154	Ü	186	║	218	┌	250	·
155	ø	187	╗	219	█	251	¹
156	£	188	╝	220	▄	252	³
157	Ø	189	¢	221	¦	253	²
158	×	190	¥	222	Ì	254	■
159	ƒ	191	┐	223	▀	255	nbsp

Figure 3, Table ASCII

Fonction	Rôle	Exemple

ord(car)	Retrouver le code ASCII/Unicode d'un caractère.	Python
		`car = "A"`
		`num = ord(car) # num = 65`

chr(num)	Retrouver le caractère correspondant à un code ASCII/Unicode.	Python
		`num = 64`
		`car = chr(num) # car = "@"`

Type chaîne de caractères

Une chaîne de caractères est formée par un ensemble de caractères.

Longueur

Une chaîne possède une **longueur** qui indique le nombre de ses caractères. On utilise la fonction `len(chaine)` pour déterminer la longueur d'une chaîne.
Une chaîne de longueur zéro est appelée **chaîne vide**. Elle est notée `""` ou `''` et `len("") → 0`.

Concaténation

Une chaîne peut être le résultat de concaténation de deux ou plusieurs chaînes `"Sa" + "mi" → "Sami"`.

Indexation

On peut **accéder aux caractères individuels** d'une chaîne à travers l'opérateur `[]`. Une chaîne peut-être **indexée** de deux façons différentes.

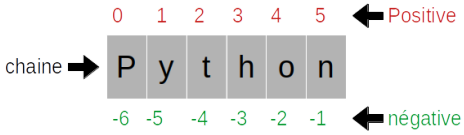


Figure 4, Indexation d'une chaîne de caractères

D'après la figure précédente, on peut retrouver le premier caractère de la `chaine` soit en utilisant **l'indexation positive** `chaine[0]` soit en utilisant **l'indexation négative** `chaine[-6]`.

Sous chaîne

On peut **extraire une partie de la chaîne** en utilisant l'opérateur `[:]`.

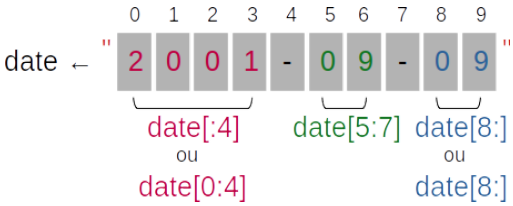


Figure 5, Extraction d'une sous-chaîne

Pour extraire une partie d'une chaîne on écrit `chaine[deb:fin]`. Cette instruction extrait les caractères compris entre les indices **deb** inclu et **fin** non inclu, **deb** est l'indice de début, et **fin** est l'indice de fin.

Recherche de position

Pour retrouver la position d'une sous-chaine `sch` dans une chaine `ch` on écrit `ch.find(sch)`, cette instruction retourne la première occurence de `sch` dans `ch`, ou `-1` si elle est introuvable.

Recherche d'existence

Pour tester l'existance d'une sous-chaîne `sch` dans une autre chaine `ch`, on écrit `sch in ch`. Cette expression renvoie un booléen : `True` / `False`.

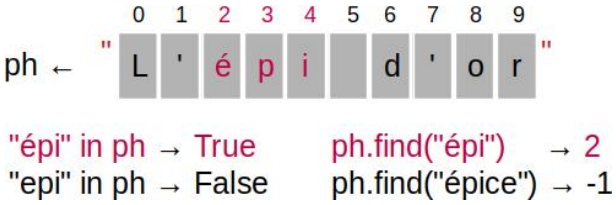


Figure 6, Recherche dans une chaîne

Effacer une sous-chaîne

Il n'existe pas une instruction Python pour **effacer une partie d'une chaîne**. Cependant, il est possible d'utiliser une concaténation de sous-chaines.
Pour effacer la partie comprise entre les indices `deb` (inclu) et `fin` (non inclu) dans une `chaine`, on écrit `chaine[:deb] + chaine[fin:]`.

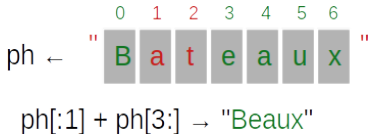


Figure 7, Effacer une sous-chaîne

Changer la casse

Python offre plusieurs méthodes pour changer la casse d'une chaine de caractère, voici quelques-unes.

- `chaine.upper()` : CONVERTIT TOUTE UNE CHAINE EN MAJUSCULES.
- `chaine.lower()` : convertit toute une chaine en minuscules.
- `chaine.capitalize()` : Met la première lettre en majuscule et les toutes autres en minuscules.
- `chaine.title()` : Convertit La Première Lettre De Chaque Mot En Majuscules.

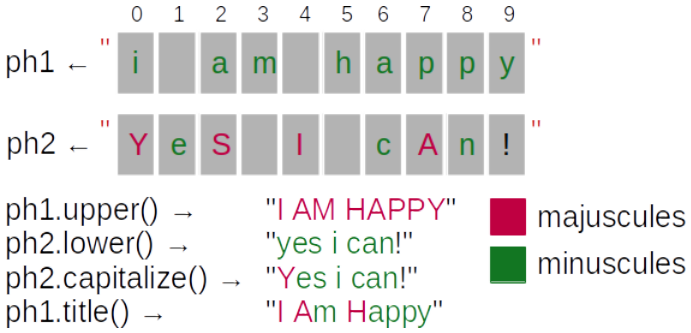


Figure 8, Modifier la casse d'une chaîne

Conversions

Une **chaîne** qui *contient une valeur numérique* peut-être transformée en **un entier** à l'aide de la fonction `int(chaine)` ou en **un réel** à l'aide de la fonction `float(chaine)`. La fonction `str(nombre)` est utilisée pour convertir un **entier** ou un **réel** en une **chaîne de caractères**.

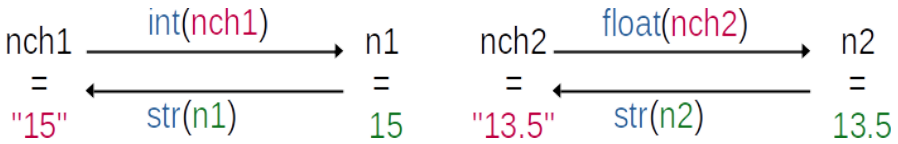


Figure 9, Conversions d'une chaîne

La conversion d'une chaîne qui ne contient pas une valeur numérique produit **une erreur d'exécution** qui peut interrompre l'exécution d'un programme pour cela on peut utiliser la fonction `chaine.isdigit()` pour tester si une chaine **contient uniquement des chiffres**.

Type entier

Le type entier couvre une partie de l'ensemble des entiers relatifs $\mathbb{Z}$, c'est à dire l'ensembles des entiers positifs 0, 1, 2, 3, etc. et des entiers négatifs -1, -2, -3, etc.
Les opérateurs possibles avec les entiers sont :

- **L'addition** `8 + 5 → 13`
- **La soustraction** `8 - 5 → 3`
- **La multiplication** `8 * 5 → 40`
- **La division réelle** `8 / 5 → 1.6`
- **Le quotient de la division entière** `8 // 5 → 1`
- **Le reste de la divion entière** `18 % 7 → 4`
- **L'exponentiation** `5 ** 3 → 125`

Voici quelques fonctions possibles avec les entiers :

- **La valeur absolue** `abs(-9) → 9`
- **Choix d'une valeur aléatoire** dans un intervalle `randint(1, 6) → une valeur au hasard dans l'intervalle [1, 6]`.
  Attention
  Pour utiliser `randint(a, b)`, il faut veiller à importer cette fonction du module `random` via l'instruction `from random import randint`.

Type réel

Le type réel couvre une partie de l'ensemble des réels $\mathbb{R}$, c'est à dire les nombres à virgule flottante.
Les opérateurs possibles avec les réels sont **les mêmes que ceux disponibles avec les entiers**, voir opérateurs avec les entiers.
Certaines fonctions avec les réels doivent être importées depuis le module `math`. Parmi ces fonctions on cite :

- **La racine carré** `sqrt(5) → 2.23606797749979`
- **La partie entière** d'un réel `trunc(6.2545) → 6`

Type booléen

Les variables de ce type peuvent prendre uniquement les deux valeurs `True` ou `False`.
On utilise les **opérateurs de comparaison** pour comparer des valeurs de mêmes types ou de types compatibles :

Opérateur	Description
<	Inférieur
<=	Inférieur ou égal
>	Supérieur
>=	Supérieur ou égal
==	égal
!=	Différent

On utilise les **opérateurs logiques** pour construire des expressions booléennes plus sophistiquées.

Opérateur	Description
not a	Négation logique, si `a` est vraie `not a` est fausse, sinon `not a` est vraie.
a and b	Et logique, `a and b` est vraie uniquement lorsque `a` est vraie et `b` est vraie.
a or b	Ou logique, `a or b` est fausse uniquement lorsque `a` est fausse et `b` est fausse, elle est vraie lorsque l'un des **opérandes** `a` ou `b` est vrai.

Tables de vérité

Opérateur unaire

a	not a
False	True
True	False

Opérateurs binaires

a	b	a and b	a or b
False	False	False	False
True	False	False	True
False	True	False	True
True	True	True	True

Priorité des opérateurs

Les opérateurs logiques sont ordonnés dans le tableau précédent selon leurs **priorités** respectives.
Il est **vivement conseillé** d'utiliser les parenthèses pour imposer l'ordre d'évaluation des opérateurs.
Exemple, l'expression `not a or b and c` est évaluée comme suit `(not a) or (b and c)`. Si on veut écrire `not ((a or b) and c)` ou `(not (a or b)) and c` , il faudra utiliser impérativement les parenthèses.

Comparaison des variables de type de numérique

Pour les entiers et les réels `a > b` si et seulement si `a - b > 0`, `a == b` si et seulement si `a - b == 0`.

Comparaison des caractères et des chaînes de caractères

La comparaison pour les **caractères** est basée sur le **code ASCII/Unicode**.

```
" " < "!" < ... < "0" < "1" < ... < "A" < "B" < ... < "a" < "b" < ...
```

Pour les **chaînes de caractères** on applique le même principe avec chacune des lettres des deux chaînes.
`"Amine" < "Eya"` puisque `"A" < "E"`
`"bassine" < "bassins"` bien qu'il soient de même longueur mais la dernière lettre `"e" < "s"`

Renforcement

Conjugaison au présent

Ecrire l'algorithme un programme qui conjuge un verbe de 1er groupe au présent.

```
Exemple de résultat
Donner un verbe de 1er groupe ? manger
Je mange
Tu manges
Il/Elle mange
Nous mangons
Vous mangez
Ils/Elles mangent
```

English calendar

Les français expriment les dates de la façon suivante : jj/mm/aaaa, alors que les anglais l'expriment d'une autre façon : mm/jj/aaaa.
Ecrire l'algorithme d'un programme qui convertit les dates du calendrier français au calendrier anglais.

```
Exemple de résultat
Date au format jj/mm/aaaa ? 24/08/1983
24/08/1983 is 08/24/1983 in english calendar.
```

Division Euclidienne

Tout nombre entier positif $n \in \mathbb{N}$ peut être exprimé sous la forme :

$$n = p.q + r$$

Avec $p > 0$, $q \geq 0$ et $0 \leq r < p$
On veut écrire l'algorithme d'un programme qui, pour deux entiers `n` et `p` donnés, calcule et affiche la valeur de `q` et `r`.

Chronomètre

Un chronomètre présente une durée au format "h:mm:ss", voir figure. On veut retrouver ce que cette valeur représente en secondes.
Ecrire l'algorithme d'un programme pour réaliser cette tâche.

```
Exemple de résultat n°1
Affichage du chronomètre ? 1:10:50
1:10:50 vaut 4250 secondes
```

```
Exemple de résultat n°2
Affichage du chronomètre ? 02:34:20
02:34:20 vaut 9260 secondes
```

Permutation de chiffres

Ecrire l'algorithme un programme qui saisit un nombre de 4 chiffres, puis permute le chiffre des centaines avec celui des milliers et le chiffre des dizaines avec celui des unités.

$$1374 \rightarrow 3147 \text{ - } 1005 \rightarrow 0150$$

Proposer **deux méthodes de résolution** :

- Une solution arithmétique.
- Une solution en utilisant les fonctions de manipulation des chaînes.

Nombres voisins

Deux nombres sont dits voisins si le dernier chiffre du premier nombre est égal au premier chiffre du second nombre.

1234 et 4200 / 67 et 730 / 730 et 67

Les deux premiers exemples présentent deux nombres voisins, le troisième d'un exemple de nombre non voisins.

Consommation de carburant

Un automobiliste surveille la consommation en carburant de sa voiture pour faire une maintenance préventive en cas de besoin.
Le matin, son réservoir initialement vide, il fait le plein de `nd` litres de carburant. A cet instant, il a parcouru `dd` kilomètres.
Le soir, l'indicateur de niveau de carburant indique `nf` litres et l'indicateur de kilométrage indique `df` kilomètres.
On demande d'écrire l'algorithme d'un programme qui saisit les données puis calcule le taux de consommation de la voiture en litres/100km.

taux = consommation / distance

Exemple de résultat n°1

```
Le matin
Niveau de carburant ? 30
Nombre de km parcourus ? 3000
Le soir
Niveau de carburant ? 10
Nombre de km parcourus ? 3200
La voiture a consommé 20 litres pour 200 km
La voiture a consommé 10.0 l/100km
```

Exemple de résultat n°2

```
Le matin
Niveau de carburant ? 10
Nombre de km parcourus ? 100
Le soir
Niveau de carburant ? 8
Nombre de km parcourus ? 127
La voiture a consommé 2 litres pour 27 km
La voiture a consommé 7.4 l/100km
```

Plein de zéros

Ecrire l'algorithme d'un programme qui saisit un nombre de 2 chiffres, puis insère autant de zéros que la différence entre ses chiffres de dizaines et d'unités.

13 → 1003 / 22 → 22 / 71 → 70000001

Prévoir deux méthodes de résoluion différentes.